

House Price Prediction

ABOUT

DATASET

This dataset is used to create the project that builds a machine learning model to predict house prices based on features such as area, number of bedrooms, bathrooms, and more. It involves data preprocessing, exploratory data analysis, model building, and evaluation.

Source: This project is inspired by a house price prediction dataset on Kaggle.

PROJECT-OVERVIEW

This machine learning project is designed to predict house prices using a dataset that includes various property features. The goal is to develop a regression model that can accurately estimate a house's price based on these characteristics.

BASED ON STEPS

1. Data collection & data Understanding
2. Data splitting
3. Model Train (SVM)
4. Model Store in pickle

LIBRARIES:

```
import pandas as pd
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split

import pickle

from sklearn.svm import SVR

from sklearn.preprocessing import StandardScaler

from sklearn.pipeline import make_pipeline
```

STEP 01 (Data Collection And Preprocessing):

The dataset is typically collected from a structured source like Kaggle, CSV files, or a real estate API. It includes features such as:

- Area
- Number of bedroom
- Number of bathrooms
- Location
- House Price (target)

Once loaded, we inspect the data using:

1. `df.head()` — preview the top rows
2. `df.info()` — understand datatypes and missing values
3. `df.describe()` — view summary statistics
4. `df.isnull().sum()` — check for nulls
5. `df.corr()` — correlation between numeric variables

This step helps to:

Detect data quality issues

Identify feature importance

Guide preprocessing and modeling decisions

STEP 01 : DATA UNDERSTANDING & PREPROCESSING

```
# ===== Using Data Class =====  
h = Data("houseprice.csv")  
df = h.load_data()  
print("First rows:\n", h.head())  
print("Missing values:\n", h.missing_values())  
print("Duplicates:", h.duplicate_rows())  
  
# Drop unused columns (example)  
h.drop_columns(["date", "street", "city", "statezip", "country"])  
  
# Fill and Encode  
h.fill_missing_mean()  
h.fill_missing_mode()  
h.encode_categorical()  
  
clean_data = h.get_data()  
print("Cleaned data shape:", clean_data.shape)  
✓ 0.0s
```

First rows:

	date	price	bedrooms	bathrooms	floors	waterfront	view	\
0	5/2/2014 0:00	313000.0	3	1.50	1.5	0	0	
1	5/2/2014 0:00	2384000.0	5	2.50	2.0	0	4	
2	5/2/2014 0:00	242000.0	2	2.00	1.0	0	0	

STEP 02 (Data Splitting):

Splits the dataset into:

Training Set (80%): To train the model

Testing Set (20%): To evaluate model performance on unseen data

Why it matters:

Prevents overfitting

Ensures fair model evaluation

STEP 02: DATA SPLITTING

```
splitter = Model(clean_data, target_column="price")
X_train, X_test, y_train, y_test = splitter.split()
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

[12] ✓ 0.0s

```
... X_train shape: (3680, 8)
    X_test shape: (920, 8)
```

STEP 03 (Model Training - Support Vector Machine):

Support Vector Regression (SVR) is a version of SVM used for predicting continuous values.

1. Feature Scaling using StandardScaler (SVM is sensitive to feature scale)
2. Model Training using SVR
3. Hyperparameter Tuning using GridSearchCV for best results

Why SVM?

Works well for both linear and non-linear relationships

Robust to overfitting (especially with proper regularization)

STEP 03 : MODEL TRAIN (SVM)

```
trainer = Trainer(X_train, y_train)
trained_model = trainer.train_svm()
print(" Model Training Complete")
```

[15] ✓ 1.6s

... Model Training Complete

STEP 04 (Model Storage - Pickle):

Once trained and tested, the model is serialized using pickle, allowing reuse without retraining.

Benefits of Pickle:

Store models for deployment

Share models across environments

Save time during inference

STEP 04 : MODEL STORE IN PICKLE

```
store_obj = Store(trained_model)
store_obj.save("svm_house_model.pkl")
loaded_model = store_obj.load("svm_house_model.pkl")
```

[21] ✓ 0.0s

... Model saved as svm_house_model.pkl
Model loaded from svm_house_model.pkl

STEP 05 (Model PREDICTION):

- Provides real-time house price estimation
- Allows testing on unseen data
- Supports integration with web apps

```
Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter variables
```

STEP 05: MODEL PREDICTION :

```
user_input = {
    'bedrooms': 3,
    'bathrooms': 2,
    'sqft_living': 1800,
    'sqft_lot': 5000,
    'floors': 1,
    'waterfront': 0,
    'view': 0,
    'condition': 3,
    'grade': 7,
    'sqft_above': 1500,
    'sqft_basement': 300,
    'yr_built': 2005,
    'yr_renovated': 0,
    'zipcode_98103': 1 # Example encoded feature
}

input_df = pd.DataFrame([user_input])
model_columns = X_train.columns
input_sample = input_df.reindex(columns=model_columns, fill_value=0)
prediction = loaded_model.predict(input_sample)
print(f"Predicted House Price: ${prediction[0]:,.2f}")
```

[20] ✓ 0.0s

... Predicted House Price: \$464,020.74